

ezr, week 2: tables, distance, gap to heaven

the story

One weekly-sized chunk of `ezr.lua`, verbatim, with glossary notes folded in and this week's exercises at the bottom.

Rows gather into tables; row 1 alone decides every column's kind and role. Then two numbers see the whole table: `distx`, the gap between two rows over the `x` columns; and `disty`, the gap from a row's goals to heaven. Sorting by `disty` is optimization with no model, no weights, no training.

Tbl I

Rows, plus the column summaries those rows built. Row 1 of any source is the header, and the header alone decides each column's kind (noir) and role: a trailing ! is the class, + or - a goal (a y column), X is ignored, the rest are the x (independent) columns.

```
function Tbl(src)
  src = iter(src)
  return adds(src, new(TBL, {rows={}, mid=nil,
                        cols=Cols(src())})) end

function Cols(names, all, x, y, klass)
  all, x, y = {}, {}, {}
  for at, s in ipairs(names) do
    all[at] = Col(s, at)
    if s:find"!$" then klass = all[at]
    elseif s:find"[+-]$" then y[#y+1] = all[at]
    elseif s:sub(-1) ~= "X" then x[#x+1] = all[at] end
  end
  return new(COLS, {names=names, all=all, x=x, y=y, klass=k
```

minkowski I

Minkowski's p -norm, folding many per-column gaps g_c (each 0..1) into one 0..1 number:

$$d = \left(\frac{1}{n} \sum_c g_c^p \right)^{1/p}$$

$p = 1$ is the Manhattan distance (all gaps count equally), $p = 2$ Euclidean; as p grows, the largest single gap dominates.

`the.p` defaults to 2.

Note the design: `minkowski` never touches the data. It takes a function `f` and calls `f(c)` per column, on demand — a higher-order, lazy style. Each caller passes its own little lambda (distx measures row gaps, disty measures gaps to heaven), and no intermediate list of gaps is ever built.

The Python analog is a generator expression, computing each term only as it is summed:

```
d = (sum(f(c)**p for c in cols) / len(cols)) ** (1/p)
```

```
function minkowski(cols,f,    d,n)
  d, n = 0, TINY
  for _, c in ipairs(cols) do n, d = n+1, d + f(c) ^ the.p
  return (d / n) ^ (1 / the.p) end
```

distx I

The gap between two rows, over the x columns only: each column measures its own 0..1 gap (`dist` in the `columnProtocol`), and `minkowski` folds them. An unknown "?" assumes the worst: symbols that might differ, do; a missing number is pushed to whichever end is further away.

```
function SYM.dist(i,a,b)
  if a == "?" and b == "?" then return 1 end
  return a ~= b and 1 or 0 end
```

```
function NUM.dist(i,a,b)
  if a == "?" and b == "?" then return 1 end
  a, b = i:norm(a), i:norm(b)
  if a == "?" then a = b > 0.5 and 0 or 1 end
  if b == "?" then b = a > 0.5 and 0 or 1 end
  return abs(a - b) end
```

```
function TBL.distx(i,row1,row2)
  return minkowski(i.cols.x, function(c)
    return c:dist(row1[c.at], row2[c.at]) end) end
```

heaven, disty, Pareto frontier, Pareto zoom effect I

The best value a goal column can hope for, in normalized 0..1 space: 0 for a minimize goal (trailing -), 1 for a maximize goal (trailing +). Decided in one line, at column birth:

The gap from a row's goals to heaven: each y column measures $|norm(v) - heaven|$, and minkowski folds them. 0 = best possible row, 1 = worst. No model, no weights, no training — sort rows by disty and the best float to the top (--disty in ezr-eg2). disty is an aggregation function, the zooming rival to frontier-chasing (see Pareto zoom effect). Later weeks make disty the thing that costs money: it reads the goal columns, and goals are labels.

“Give me the fruitful error any time, full of seeds, bursting with its own corrections. You can keep your sterile truth for yourself.” — Vilfredo Pareto

With many goals there is rarely one best row — lighter cars brake worse. Row **a** dominates **b** if **a** is at least as good on

```
function TBL.disty(i,row)
  if i.model and row[i.cols.y[1].at] == "?" then
    i:label(row) end
  return minkowski(i.cols.y, function(y)
    return abs(y:norm(row[y.at]) - y.heaven) end
```

exercises I

Exercises, week 2

1. Port this page's code to Python, wiring each demo of `ezr-eg2` (`--distx` `--disty` `--laws`) to a `test_` function.
2. On `auto93`, sort all rows by `disty` and print the top and bottom three. Do the best rows look best? (If not, see exercise 5 of week 1.)
3. `--laws` probes distance laws at random. State the four laws. Which one fails for the “?”-handling here, and why is that a price worth paying?
4. Set `the.p=1`, then 4, then 8. How does the `disty` ranking change? What is `minkowski` converging to as `p` grows?
5. `distx` never reads a goal; `disty` reads nothing else. Why does that split matter when labels cost money?